Тестовое техническое задание кандидата на вакансию «Junior Системный аналитик».

Обращаю ваше внимание на то, что моя специальность - Бизнес-аналитик. Задания 2 и 3 являются для меня непрофильными.
Тем не менее, я постарался ознакомиться (пока поверхностно) с неизвестными темами и выполнить задание, потому что мне интересно изучать смежные направления.
Приму с пониманием ваш отказ из-за недостатка знаний в требуемых вопросах.
Буду рад сотрудничеству или хотя бы комментарию к выполненному заданию.

Задание 1: Анализ требований

Раздел ТЗ: Функционал корзины

1. Пользователь может добавить в корзину от 1 до 10 единиц одного товара.
2. Пользователь может изменить количество каждого товара в корзине не менее, чем до 1-го. Для удаления товара из корзины используется отдельная кнопка.
3. В корзине может находиться не более 5 различных товаров.
4. Суммарное количество всех товаров в корзине не может превышать 20 штук.
5. Товары в корзине могут быть разные.
6. При попытке добавить товар, превышающий лимиты, система показывает сообщение: "Лимит корзины превышен".
7. Цена на продукт фиксируется на момент добавления в корзину и не меняется.
8. На странице корзины отображается список товаров, их количество, цена за единицу и общая стоимость позиции.
9. Если пользователь уменьшает количество товара до 0, товар удаляется из корзины.
10. В корзине может быть реклама других продуктов.
11. Реклама товаров в корзине должна быть каждый будний день по утрам и вечерам.
13. Если цена на товар изменилась в каталоге, система должна автоматически обновить ее в корзине у всех пользователей.

Разбор раздела ТЗ Функционал корзины

1. Выявлено отсутствие следующей информации:
   - не указан уровень доступа к разделу;
   - не указаны связи сущностей;
   - не указан способ перехода в раздел.

2. Выявлены логические противоречия и недочеты:

Функционал	Противоречие	Недочет
1. Пользователь может добавить в корзину от 1 до *10* единиц одного товара.		*Возможно, достаточно будет ограничить максимальное число цифрой 9. Цифра 10 – двузначное число и потребует расширение пространства для отображения*
2. Пользователь может изменить количество каждого товара в корзине не менее, чем до 1-го. *Для удаления товара из корзины используется отдельная кнопка.*	*Противоречит п.9 Следует выбрать один из вариантов: удалить по кнопке или при снижении количества до «0»*	
5. *Товары в корзине могут быть разные.*		*Данное утверждение после п.3 не имеет смысла (дублирует информацию)*
7. Цена на продукт фиксируется на момент добавления в корзину *и не меняется.*	*Противоречит п.13*	*Желательно предусмотреть вариант длительного нахождения товара в корзине и корректировки цены при ее изменении*
7. Цена на *продукт* фиксируется на момент добавления в корзину и не меняется.		*Речь идет об экземпляре сущности, добавленной в корзину. По ТЗ это ТОВАР, а не ПРОДУКТ*
8. На странице корзины отображается *список товаров, их количество,* цена за единицу и общая стоимость позиции.		*Следует конкретизировать какое количество отображается. Если дальше указано, что должна отображаться цена за единицу, значит ли это что речь идет о количестве каждого вида товара? Дополнительно можно добавить отображение общей суммы заказа.*

9. *Если пользователь уменьшает количество товара до 0, товар удаляется из корзины.*	*Противоречит п.2 Следует выбрать один из вариантов: удалить по кнопке или при снижении количества до «0»*	
10. В корзине *может* быть реклама других продуктов.		*Когда слово «может» применяется в контексте, например, со словом «пользователь» — это означает, что у пользователя есть выбор делать это или нет. Реклама не может решать быть ей или не быть. Возможно задание разработчику должно предусматривать наличие места под размещение рекламы и связь со сторонними системами*
11. Реклама товаров в корзине должна *быть* каждый будний день по утрам и вечерам.		*Вряд ли предполагается, что между утром и вечером рекламы нет. Скорее допущена опечатка и вместо «быть» следует написать «обновляться»*
13. Если цена на товар изменилась в каталоге, *система должна автоматически обновить ее в корзине у всех пользователей.*	*Противоречит п.7*	*Желательно предусмотреть вариант длительного нахождения товара в корзине и корректировки цены при ее изменении*

Предлагаемый вариант раздела ТЗ Функционал корзины:

Функционал управления корзиной доступен только пользователям с ролью “Покупатель”, «Менеджер клиентского сервиса” и «Администратор» и обеспечивает процесс создания сущности Заказ формируемый из сущности Товар.

Раздел Корзина доступен авторизованным пользователям с ролью “Покупатель” при выборе элемента “Ваша корзина” на главном экране ЛК.

Товар в корзине всегда связан с Каталогом - к одному Каталогу может принадлежать множество Товаров, один Товар всегда принадлежит одному Каталогу.

1. Пользователь может добавить Товар в Корзину выбрав элемент «Добавить в корзину» на карточке Товара из Каталога;
2. Пользователь может добавить в Корзину от 1 до 10 единиц одного Товара;
3. Пользователь может изменить количество каждого Товара в Корзине не менее, чем до 1-го;
4. Для удаления Товара из Корзины используется отдельная кнопка;
5. Товары в Корзине могут быть разные;
6. Должна проверяться уникальность добавляемого Товара - если при добавлении найден объект с полностью соответствующей комбинацией атрибутов, сохранение Товара в Корзине не осуществляется, отображается системное сообщение “Товар уже в корзине”;
7. В Корзине может находиться не более 5 различных Товаров;
8. Суммарное количество всех Товаров в Корзине не может превышать 20 штук;
9. При попытке добавить Товар, превышающий лимиты, система показывает сообщение: "Лимит корзины превышен";
10. Цена на Товар фиксируется на момент добавления в Корзину и не меняется в течение 24 часов;
11. После истечения 24 часов, если Заказ не был создан, появится сообщение «Обновите информацию о товаре» с предложением «Обновить» / «Удалить товар». Если цена на Товар изменилась в Каталоге, система должна автоматически обновить ее в Корзине;
12. На странице Корзины отображается список Товаров, для каждого Товара количество, цена за единицу и общая стоимость позиции, а также общая сумма Заказа.
13. В Корзине будет реклама других продуктов. Реклама товаров в Корзине должна обновляться каждый будний день по утрам и вечерам из модуля «Реклама» по ссылке…

Задание 2: проектирование API

1. REST API запрос

HTTP метод: `GET`

URL: `/api/v1/stores-screen`

Заголовки (Headers):

```json
{
  "Content-Type": "application/json",
  "Authorization": "Bearer <user_token>",
  "Accept-Language": "ru-RU"
}
```

Параметры запроса (Query Parameters):

- `user_id` — идентификатор пользователя (обязательный);
- `screen_type` — тип экрана (опционально, по умолчанию `main`);
- `include_promotions` — флаг включения акций (опционально, по умолчанию `true`).

Пример полного URL с параметрами:

```
GET /api/v1/stores-screen?user_id=12345&screen_type=main&include_promotions=true
```

2. Пример ответа REST API (JSON)

```json
{
  "status": "success",
  "timestamp": "2026-04-23T12:30:45Z",
  "data": {
    "screen_title": "Выберите магазин",
    "stores": [
      {
        "id": "store_001",
        "name": "METPO",
        "logo_url": "https://cdn.example.com/logos/metro.png",
        "external_link": " https://www.metro.ru/example-delivery/",
        "description": "Ближайшая доставка",
        "start_of_delivery": 21,
        "finish_of_delivery": 23,
      },
```

```
      {
        "id": "store_002",
        "name": "АШАН",
        "logo_url": "https://cdn.example.com/logos/auchan.png",
         "external_link": " https://www.auchan.ru/example-delivery/",
        "description": "Ближайшая доставка",
        "start_of_delivery": 18,
        "finish_of_delivery": 20,
      },

      {
        "id": "store_003",
        "name": "ВкусВилл",
        "logo_url": "https://cdn.example.com/logos/vkusvill.png",
        "external_link": " https://www.vkusvill.ru/example-delivery/",
        "description": "Быстрая доставка",
        "min_time_of_delivery": "от 20 минут",
        "max_time_of_delivery": "до 60 минут",
      },
      {
        "id": "store_004",
        "name": "ВИКТОРИЯ",
        "logo_url": "https://cdn.example.com/logos/viktoria.png",
        "external_link": " https://www. victoria-group.ru/example-delivery/",
        "description": "Ближайшая доставка",
        "start_of_delivery": 17,
        "finish_of_delivery": 19,
      }
    ],
   }
  "total_stores": 4,
}
```


3. Описание структуры ответа

Корневые поля:

- `status` — статус выполнения запроса (`success`/`error`);
- `timestamp` — время формирования ответа;
- `data` — основные данные экрана.

Поля внутри `data`:

- `screen_title` — заголовок экрана;
- `subtitle` — подзаголовок (при наличии);
- `stores` — массив объектов магазинов;
- `total_stores` — общее количество доступных магазинов;
- `filters_available` — доступные фильтры для сортировки (при наличии).

Структура объекта магазина (`stores`):

- `id` — уникальный идентификатор;
- `name` — название магазина;
- `logo_url` — ссылка на логотип;
- `external_link` — **ссылка для перехода на внешний ресурс** (открывается при кл ике на плашку);
- `description` — краткое описание;
- `start_of_delivery` — начало промежутка доставки;
- `finish_of_delivery` — окончание промежутка доставки;
- `min_time_of_delivery` — минимальное время доставки;
- `max_time_of_delivery` — максимальное время доставки.

4. Поведение при клике

При клике на плашку магазина выполняется следующее:

1. Берётся значение поля `external_link` из объекта магазина.
2. Открывается внешняя ссылка в:
   - новом окне браузера (на веб-версии);
   - встроенном веб-вью (в мобильном приложении);
   - либо запускается внешнее приложение, если ссылка поддерживает deep linking.

Задание 3: архитектура

Верхнеуровневая архитектура системы отправки PUSH-уведомлений

Компоненты системы

1. **Клиентское приложение** (мобильное/веб):
   - запрашивает у пользователя разрешение на получение уведомлений;
   - регистрируется в сервисе PUSH-уведомлений (получает уникальный токен/подписку);
   - передаёт токен на сервер приложения.

2. **Сервер приложения** (Backend):
   - хранит токены устройств пользователей в БД;
   - формирует содержимое PUSH-уведомлений;
   - инициирует отправку через соответствующий сервис;
   - ведёт логирование и статистику доставки.

3. **Платформенные PUSH-сервисы**:
   - **APNs** (Apple Push Notification service) — для iOS/macOS;
   - **FCM** (Firebase Cloud Messaging) — для Android;
   - **HPK** (Huawei Push Kit) — для устройств Huawei без Google Services;
   - **RuStore/Aurora** — российские аналоги для Android;
   - для веб-приложений — поддержка через **Service Worker** и **Push API**.

4. **Сервис управления уведомлениями** (опционально):
   - централизованная маршрутизация сообщений;
   - каскадирование каналов (переключение на резервные сервисы при сбоях);
   - аналитика и мониторинг доставки;
   - сегментация аудитории и A/B-тестирование.

5. **База данных**:
   - хранение токенов устройств;
   - история отправленных уведомлений;
   - настройки уведомлений пользователей (разрешения, категории).

6. **Система триггеров**:
   - отслеживает события, требующие отправки уведомлений (новые сообщения, акции, напоминания и т.д.);
   - запускает процесс формирования и отправки PUSH.

Поток данных (алгоритм работы)

1. **Регистрация устройства**:
   - пользователь запускает приложение и даёт разрешение на уведомления;
   - приложение получает уникальный токен от платформенного сервиса (APNs/FCM и т.д.);
   - токен передаётся на сервер приложения и сохраняется в БД.

2. **Инициирование отправки**:
   - система триггеров фиксирует событие (например, новое сообщение в чате);

- сервер приложения определяет целевую аудиторию (пользователи, которым нужно отправить уведомление);
- формируется payload уведомления (текст, заголовок, deeplink, иконки и т.д.).

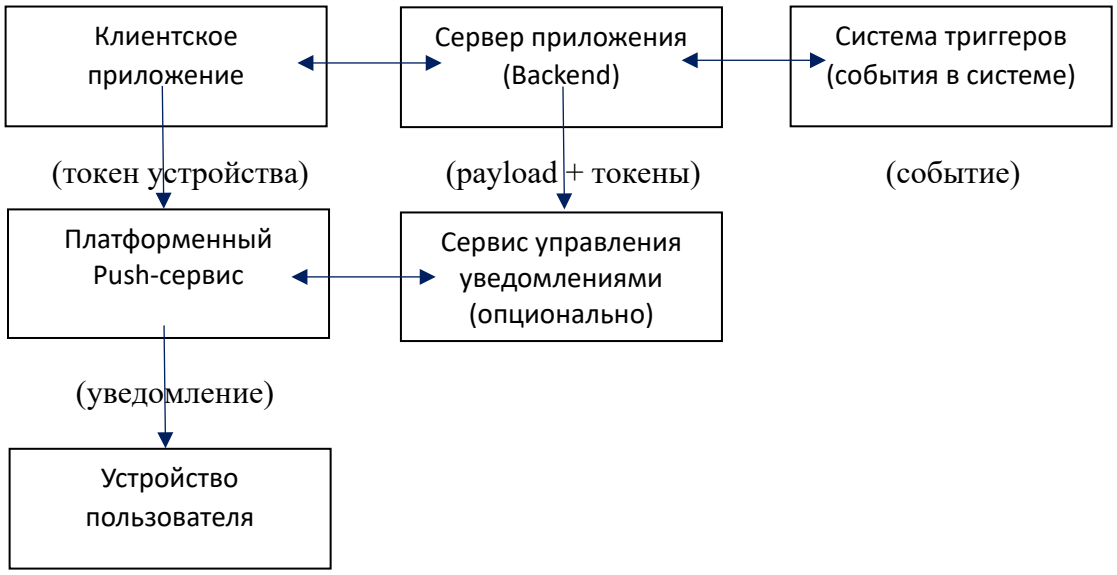
3. **Отправка через платформенный сервис**:
   - сервер отправляет HTTP POST-запрос на API сервиса (APNs, FCM и т.д.) с payload и токенами устройств;
   - сообщение шифруется (например, по протоколу VAPID для веб-уведомлений);
   - сервис подтверждает приём сообщения в обработку.

4. **Доставка на устройство**:
   - платформенный сервис маршрутизирует сообщение на целевое устройство через постоянное соединение;
   - операционная система получает уведомление и отображает его через нативные механизмы (Notification Center, Action Center и т.д.).

5. **Взаимодействие с пользователем**:
   - пользователь видит уведомление на экране блокировки/в панели уведомлений;
   - при клике на уведомление приложение открывается (или запускается) и переходит к нужному разделу (если указан deeplink).

6. **Обратная связь и аналитика** (опционально):
   - сервер получает статусы доставки/прочтения (если поддерживается сервисом);
   - данные логируются и используются для аналитики (конверсия, время открытия и т.д.).

Схема архитектуры (текстовое представление)



Ключевая особенность

- **Масштабируемость**: архитектура Pub/Sub (Publish/Subscribe) позволяет обрабатывать миллионы устройств.

Рекомендации

- **Безопасность**: используйте шифрование (VAPID для веб, встроенные механизмы APNs/FCM).
- **Надёжность**: реализуйте каскадирование (резервные каналы) и повторные попытки отправки при ошибках.
- **Аналитика**: отслеживайте метрики (доставка, открытия, конверсия) для оптимизации кампаний.
- **Соответствие регуляциям**: соблюдайте GDPR, 152-ФЗ и другие нормы (отписка, согласие на уведомления).
- **Кастомизация**: поддерживайте rich-уведомления (изображения, кнопки действий, deeplink).